AIAC-ASSIGNMENT-17

NAME : GOLI NITHYA **BATCH:32** **HNO:2303A52066**

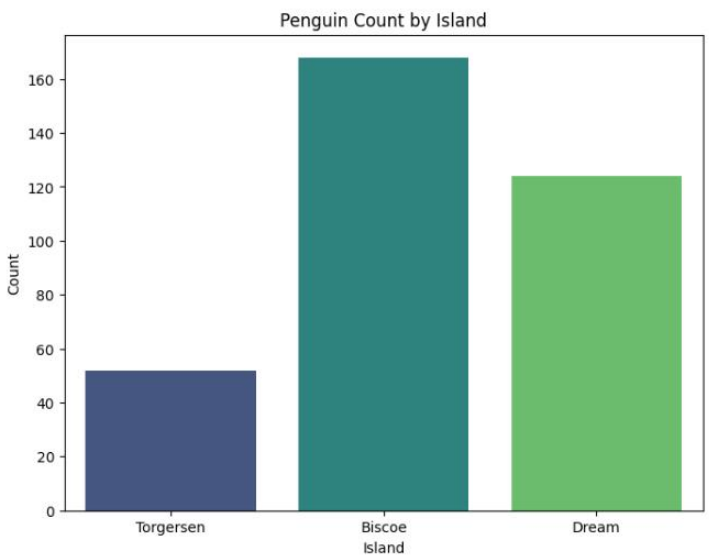
1.Perform EDA on Penguin dataset(from seaborn)

```python
import torch
import seaborn as sns
import pandas as pd

if torch.cuda.is_available():
    print("CUDA is available! You can use your GPU.")
else:
    print("CUDA is not available. Running on CPU.")

# Load the penguins dataset
penguins_df = sns.load_dataset('penguins')

# Display the first 5 rows of the dataset
display(penguins_df.head())
```

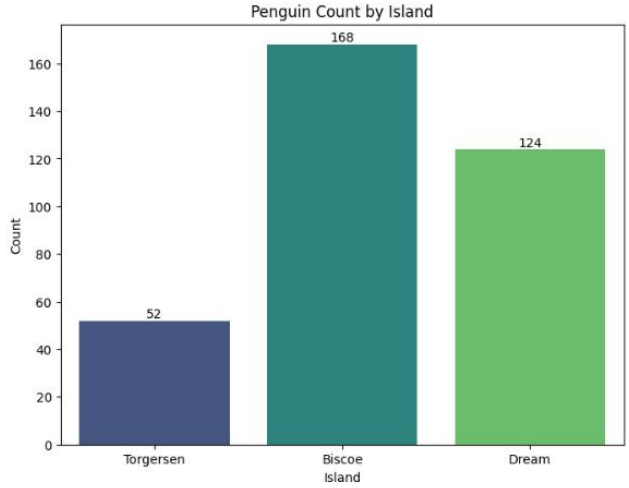
```
CUDA is available! You can use your GPU.
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

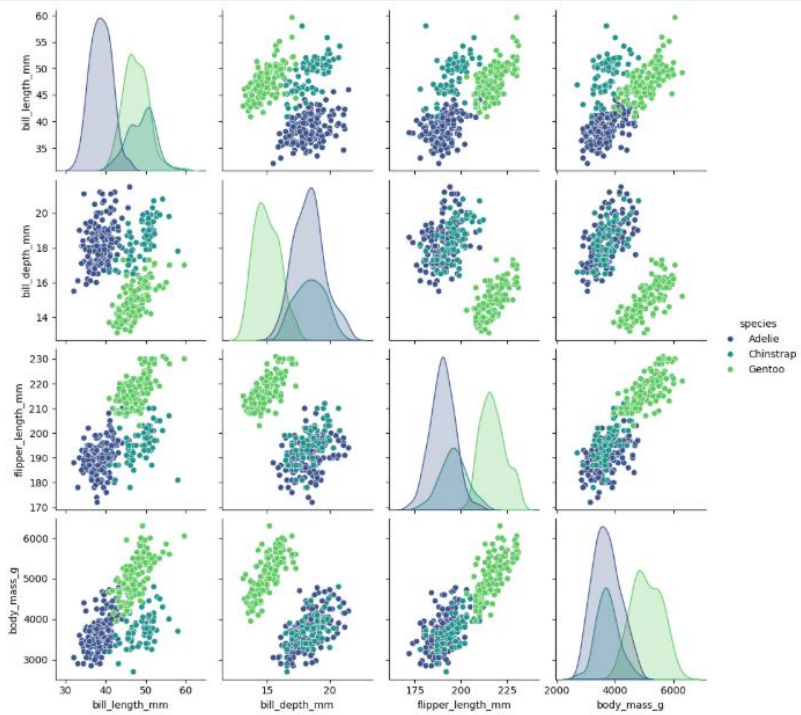
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a countplot for the 'island' column
plt.figure(figsize=(8, 6))
sns.countplot(data=penguins_df, x='island', hue='island', palette='viridis', legend=False)
plt.title('Penguin Count by Island')
plt.xlabel('Island')
plt.ylabel('Count')
plt.show()
```



```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a countplot for the 'island' column
plt.figure(figsize=(8, 6))
ax = sns.countplot(data=penguins_df, x='island', hue='island', palette='viridis', legend=False)
plt.title('Penguin Count by Island')
plt.xlabel('Island')
plt.ylabel('Count')

# Add count labels on top of each bar
for container in ax.containers:
    ax.bar_label(container, fmt='%d')

plt.show()
```




```python
import seaborn as sns
import matplotlib.pyplot as plt

# Create a pairplot of the penguins dataset
# Using 'species' as hue to differentiate species in the plots
sns.pairplot(penguins_df, hue='species', palette='viridis')
plt.suptitle('Pairplot of Penguin Features by Species', y=1.02) # Add a title to the whole figure
plt.show()
```



2: Perform EDA on HealthExp dataset(seaborn)

```
import sys
!{sys.executable} -m pip install seaborn
```

```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.pos
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn)
```

```
import seaborn as sns
import pandas as pd

# Load the HealthExp dataset
health_exp_df = sns.load_dataset('healthexp')

# Display the first 5 rows of the DataFrame
display(health_exp_df.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

```
display(health_exp_df)
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9
...	...	...	...	...
269	2020	Germany	6938.983	81.1
270	2020	France	5468.418	82.3
271	2020	Great Britain	5018.700	80.4
272	2020	Japan	4665.641	84.7
273	2020	USA	11859.179	77.0

274 rows × 4 columns

Next steps: Generate code with health_exp_df | New interactive sheet

All Columns in the DataFrame

```
display(health_exp_df.columns)
```

```
Index(['Year', 'Country', 'Spending_USD', 'Life_Expectancy'], dtype='object')
```

Statistical Summary of the DataFrame

```
display(health_exp_df.describe())
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

Missing Values in the DataFrame

```
# Calculate missing values
missing_values = health_exp_df.isnull().sum()

# Display missing values
display(missing_values)
```

	0
Year	0
Country	0
Spending_USD	0
Life_Expectancy	0

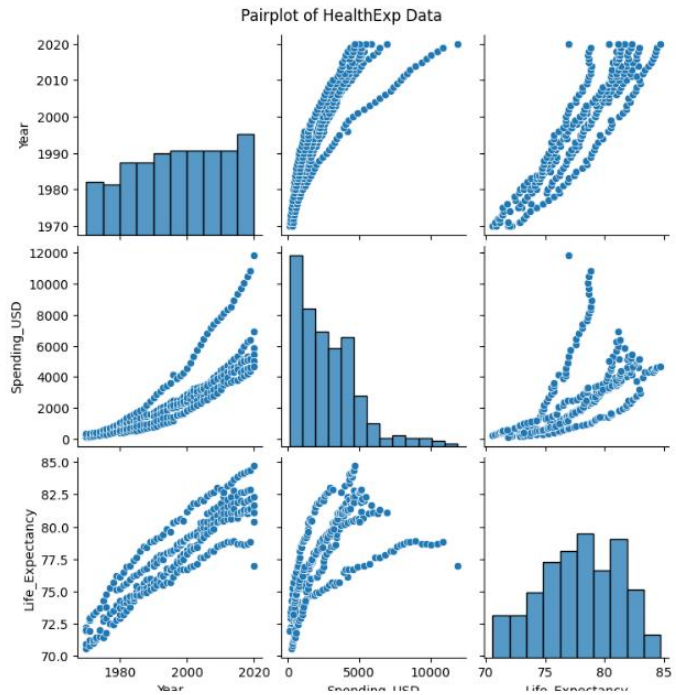
dtype: int64

Bar Chart of Missing Values

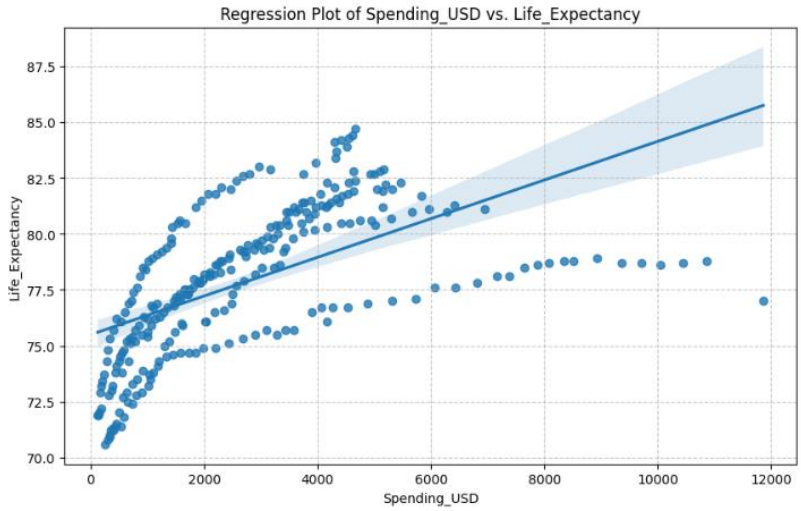
```
import matplotlib.pyplot as plt

# Filter out columns with no missing values for the chart
missing_values_to_plot = missing_values[missing_values > 0]

if not missing_values_to_plot.empty:
    plt.figure(figsize=(10, 6))
    missing_values_to_plot.plot(kind='bar', color='skyblue')
    plt.title('Number of Missing Values per Column')
    plt.xlabel('Columns')
    plt.ylabel('Number of Missing Values')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print('No missing values found in the DataFrame.')
```

```
No missing values found in the DataFrame.
```

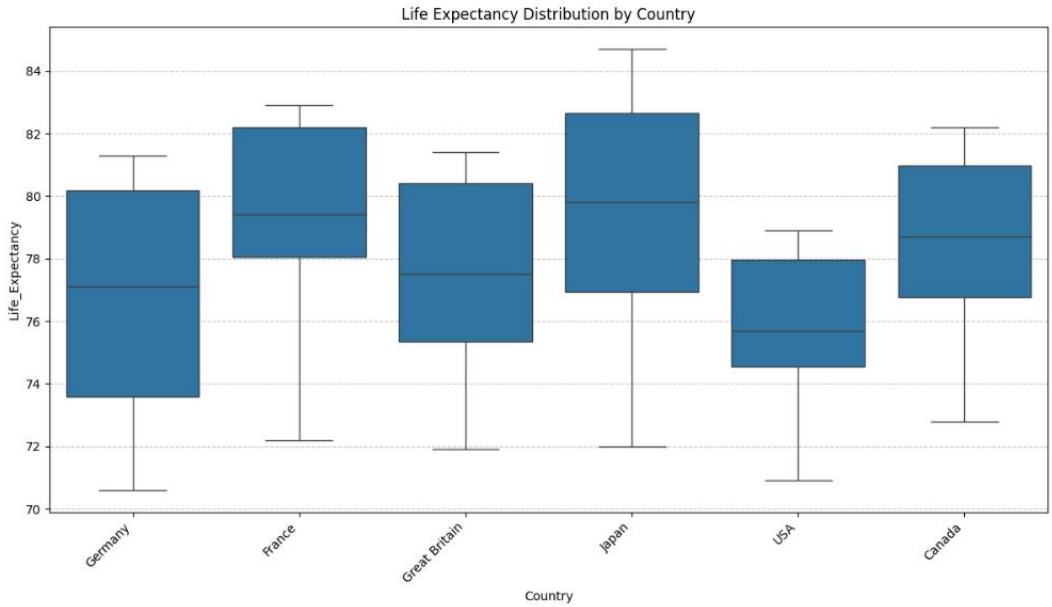
```
import seaborn as sns
import matplotlib.pyplot as plt

# Pairplot: Visualize pairwise relationships in the dataset
sns.pairplot(health_exp_df)
plt.suptitle('Pairplot of HealthExp Data', y=1.02) # Adjust suptitle position
plt.show()
```



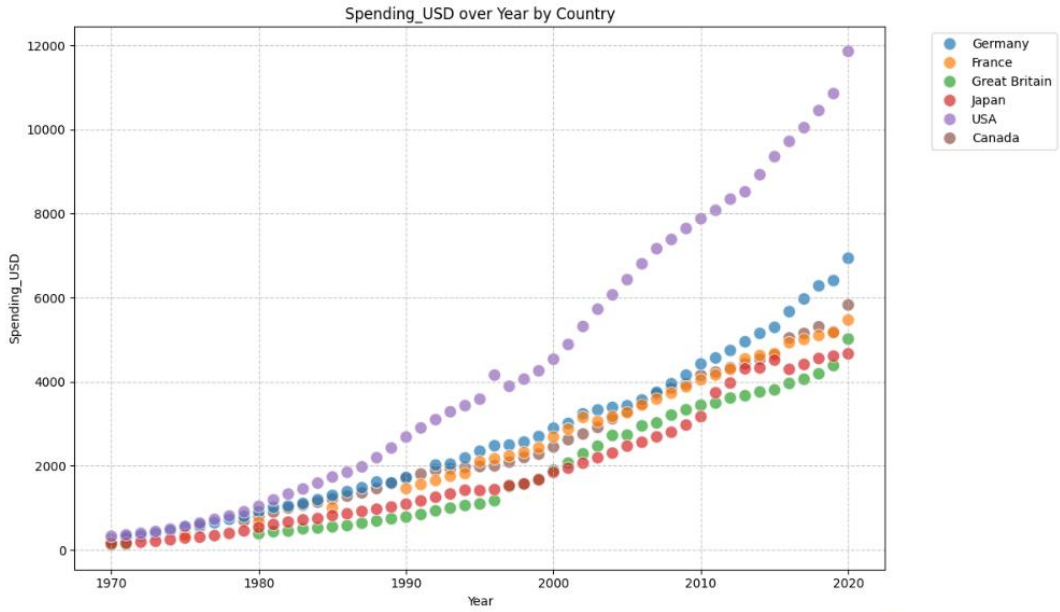
```python
# Regplot: Visualize the linear relationship between Spending_USD and Life_Expectancy
plt.figure(figsize=(10, 6))
sns.regplot(x='Spending_USD', y='Life_Expectancy', data=health_exp_df)
plt.title('Regression Plot of Spending_USD vs. Life_Expectancy')
plt.xlabel('Spending_USD')
plt.ylabel('Life_Expectancy')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```




```python
# Boxplot: Visualize the distribution of Life_Expectancy across Countries
plt.figure(figsize=(12, 7))
sns.boxplot(x='Country', y='Life_Expectancy', data=health_exp_df)
plt.title('Life Expectancy Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Life_Expectancy')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



```python
# Scatterplot: Visualize the relationship between Year and Spending_USD, with Country as hue
plt.figure(figsize=(12, 7))
sns.scatterplot(x='Year', y='Spending_USD', hue='Country', data=health_exp_df, s=100, alpha=0.7)
plt.title('Spending_USD over Year by Country')
plt.xlabel('Year')
plt.ylabel('Spending_USD')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



4: Perform following task on given dataset (Train the model)

[1]
✓ 9s

```python
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Check for CUDA availability and set the device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

[2]
✓ 3s

```python
# Define transformations for the dataset
transform = transforms.Compose([
    transforms.ToTensor(), # Convert images to PyTorch tensors
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize pixel values (mean, std for MNIST)
])

# Load the MNIST training and test datasets
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

# Create data loaders for batching
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)

print(f"Number of training samples: {len(train_dataset)}")
print(f"Number of testing samples: {len(test_dataset)}")
print(f"Image shape: {train_dataset[0][0].shape}") # Example: (1, 28, 28) for grayscale images
print(f"Number of classes: {len(train_dataset.classes)}")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 481kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.45MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 16.5MB/s]Number of training samples: 60000
Number of testing samples: 10000
Image shape: torch.Size([1, 28, 28])
Number of classes: 10
```

Let's visualize some sample images from the training dataset with their labels.

```python
import torch.nn as nn
from torchvision import models

class EfficientNet_MNIST(nn.Module):
    def __init__(self, num_classes=10):
        super(EfficientNet_MNIST, self).__init__()
        # Load a pre-trained EfficientNet model (e.g., EfficientNet_b0)
        self.efficientnet = models.efficientnet_b0(pretrained=True)

        # Modify the first convolutional layer to accept single-channel MNIST images
        # Original EfficientNet expects 3 input channels (RGB).
        # We will take the first channel's weights, repeat them 3 times, and average them,
        # or simply create a new conv layer that takes 1 input channel.
        # A simpler way is to replace the first conv layer.
        self.efficientnet.features[0][0] = nn.Conv2d(1, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)

        # Replace the classifier head for MNIST (10 classes)
        num_ftrs = self.efficientnet.classifier[1].in_features
        self.efficientnet.classifier[1] = nn.Linear(num_ftrs, num_classes)

    def forward(self, x):
        return self.efficientnet(x)

# Instantiate the model and move it to the device
model = EfficientNet_MNIST(num_classes=10).to(device)
print(f"EfficientNet model designed and moved to {device}.")

# Print a summary of the model (optional, but good for verification)
# try:
#     from torchsummary import summary
#     summary(model, (1, 28, 28)) # Input image size for MNIST (1 channel, 28x28)
# except ImportError:
#     print("Install torchsummary for detailed model summary: pip install torchsummary")
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated sinc
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwightman-7f5810bc.pt
100%|██████████| 20.5M/20.5M [00:00<00:00, 162MB/s]
EfficientNet model designed and moved to cuda.
```

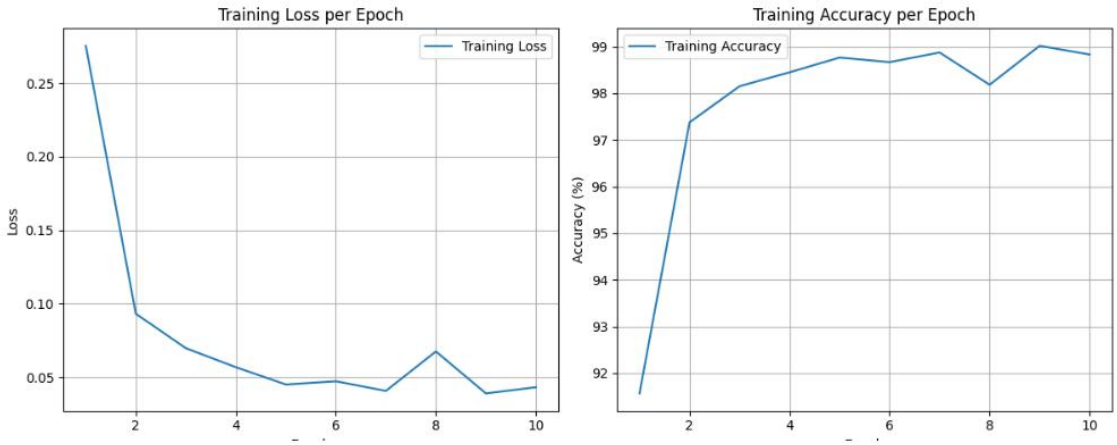
```python
import torch.optim as optim

# Define Loss Function and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training the Model
num_epochs = 10

train_losses = []
train_accuracies = []

print("Starting model training...")
for epoch in range(num_epochs):
    model.train() # Set the model to training mode
    running_loss = 0.0
    correct_predictions = 0
    total_predictions = 0

    for i, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad() # Zero the parameter gradients
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward() # Backward pass
        optimizer.step() # Optimize

        running_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total_predictions += labels.size(0)
        correct_predictions += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_accuracy = 100 * correct_predictions / total_predictions

    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_accuracy)

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}, Accuracy: {epoch_accuracy:.2f}%')

print("Finished Training!")
```

```
Starting model training...
Epoch [1/10], Loss: 0.2753, Accuracy: 91.56%
Epoch [2/10], Loss: 0.0931, Accuracy: 97.38%
Epoch [3/10], Loss: 0.0697, Accuracy: 98.15%
Epoch [4/10], Loss: 0.0568, Accuracy: 98.45%
Epoch [5/10], Loss: 0.0450, Accuracy: 98.77%
Epoch [6/10], Loss: 0.0472, Accuracy: 98.67%
Epoch [7/10], Loss: 0.0407, Accuracy: 98.88%
Epoch [8/10], Loss: 0.0675, Accuracy: 98.18%
Epoch [9/10], Loss: 0.0390, Accuracy: 99.02%
Epoch [10/10], Loss: 0.0432, Accuracy: 98.83%
Finished Training!
```

```
model.eval() # Set the model to evaluation mode
correct_predictions = 0
total_predictions = 0

with torch.no_grad(): # Disable gradient calculation during inference
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        total_predictions += labels.size(0)
        correct_predictions += (predicted == labels).sum().item()

test_accuracy = 100 * correct_predictions / total_predictions
print(f'Accuracy of the model on the {len(test_dataset)} test images: {test_accuracy:.2f}%')
```

```
Accuracy of the model on the 10000 test images: 99.34%
```

```
plt.figure(figsize=(12, 5))

# Plot training loss
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(range(1, num_epochs + 1), train_losses, label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss per Epoch')
plt.legend()
plt.grid(True)

# Plot training accuracy
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(range(1, num_epochs + 1), train_accuracies, label='Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.title('Training Accuracy per Epoch')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
# Function to unnormalize images for visualization
def imshow(img):
    img = img / 2 + 0.5  # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# Get a batch of training images
dataiter = iter(train_loader)
images, labels = next(dataiter)

# Show images
plt.figure(figsize=(10, 4))
imshow(torchvision.utils.make_grid(images[:8])) # Display first 8 images

# Print labels
print(' '.join(f'{train_dataset.classes[labels[j]]:5s}' for j in range(8)))
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.28789353..1.9107434
```

```
3 - three 6 - six 3 - three 8 - eight 1 - one 7 - seven 3 - three 1 - one
```

6 – Sales Data Preprocessing

2. Normalize sales amount column

```
if df is not None and 'Sales Amount' in df.columns:
    scaler = MinMaxScaler()
    df['Normalized_Sales_Amount'] = scaler.fit_transform(df[['Sales Amount']])
    print("Sales amount column normalized. Displaying head with new column:")
    display(df[['Sales Amount', 'Normalized_Sales_Amount']].head())
elif df is not None:
    print("No 'Sales Amount' column found for normalization.")
```

3. Detect and handle outliers in dataset

```
if df is not None and 'Sales Amount' in df.columns:
    # Using IQR method for outlier detection
    Q1 = df['Sales Amount'].quantile(0.25)
    Q3 = df['Sales Amount'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df['Sales Amount'] < lower_bound) | (df['Sales Amount'] > upper_bound)]

    print(f"Number of outliers detected in 'Sales Amount': {len(outliers)}")
    if not outliers.empty:
        print("Sample of detected outliers:")
        display(outliers.head())

        # Handling outliers: Cap values at the bounds (you can also choose to remove or impute)
        df['Sales Amount Handled'] = np.where(
            df['Sales Amount'] < lower_bound, lower_bound,
            np.where(df['Sales Amount'] > upper_bound, upper_bound, df['Sales Amount'])
        )
        print("Outliers handled by capping. Displaying original vs handled sales amount:")
        display(df[['Sales Amount', 'Sales Amount Handled']].head())
    else:
        print("No significant outliers detected in 'Sales Amount' based on IQR method.")
elif df is not None:
    print("No 'Sales Amount' column to detect and handle outliers.")
```

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Load the dataset from the uploaded CSV file
try:
    df = pd.read_csv('sales_data.csv')
    print("Dataset loaded successfully. Displaying first 5 rows:")
    display(df.head())
except FileNotFoundError:
    print("Error: 'sales_data.csv' not found. Please ensure you have uploaded the file to your Colab environment.")
    df = None # Set df to None to prevent further errors if file is not found
```

```
Error: 'sales_data.csv' not found. Please ensure you have uploaded the file to your Colab environment.
```

1. Convert date columns into datetime format and create new features

```
if df is not None:
    # Assuming 'Date' is the column containing date information
    if 'Date' in df.columns:
        df['Date'] = pd.to_datetime(df['Date'])
        df['Month'] = df['Date'].dt.month
        df['Year'] = df['Date'].dt.year
        df['DayOfWeek'] = df['Date'].dt.dayofweek # Monday=0, Sunday=6
        print("Date columns converted and new features created. Displaying info and head:")
        display(df.info())
        display(df.head())
    else:
        print("No 'Date' column found in the dataset for date conversion and feature creation.")
```